

MESSAGE PASSING SYSTEMS

Message passing systems are probably the most common platforms for parallel processing today.

7.1 Overview

Traditionally, shared-memory hardware has been extremely expensive, with a typical system costing hundreds of thousands of dollars. Accordingly, the main users were for very large corporations or government agencies, with the machines being used for heavy-duty server applications, such as for large databases and World Wide Web sites. The conventional wisdom is that these applications require the efficiency that good shared-memory hardware can provide.

But the huge expense of shared-memory machines led to a quest for high-performance message-passing alternatives, first in hypercubes and then in networks of workstations (NOWs).

The situation changed radically around 2005, when “shared-memory hardware for the masses” became available in dual-core commodity PCs. Chips of higher core multiplicity are commercially available, with a decline of price being inevitable. Ordinary users will soon be able to afford shared-memory machines featuring dozens of processors.

Yet the message-passing paradigm continues to thrive. Many people believe it is more amenable to writing really fast code, and the advent of **cloud computing** has given message-passing a big boost. In addition, many of the world’s very fastest systems (see www.top500.org for the latest list) are in fact of the message-passing type.

In this chapter, we take a closer look at this approach to parallel processing.

7.2 A Historical Example: Hypercubes

A popular class of parallel machines in the 1980s and early 90s was that of **hypercubes**. Intel sold them, for example, as did a subsidiary of Oracle, nCube. A hypercube would consist of some number of ordinary Intel processors, with each processor having some memory and serial I/O hardware for connection to its “neighbor” processors.

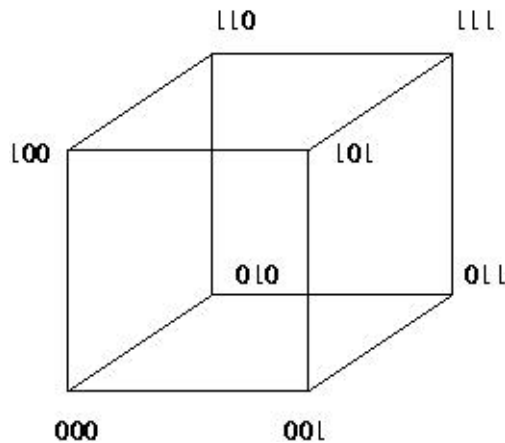
Hypercubes proved to be too expensive for the type of performance they could achieve, and the market was small anyway. Thus they are not common today, but they are still important, both for historical reasons (in the computer field, old techniques are often recycled decades later), and because the algorithms developed for them have become quite popular for use on general machines. In this section we will discuss architecture, algorithms and software for such machines.

7.2.1 Definitions

A **hypercube** of dimension d consists of $D = 2^d$ **processing elements** (PEs), i.e. processor-memory pairs, with fast serial I/O connections between neighboring PEs. We refer to such a cube as a **d-cube**.

The PEs in a d -cube will have numbers 0 through $D-1$. Let $(c_{d-1}, \dots, c_0)$ be the base-2 representation of a PE's number. The PE has fast point-to-point links to d other PEs, which we will call its **neighbors**. Its i th neighbor has number $(c_{d-1}, \dots, 1 - c_{i-1}, \dots, c_0)$.¹

For example, consider a hypercube having $D = 16$, i.e. $d = 4$. The PE numbered 1011, for instance, would have four neighbors, 0011, 1111, 1001 and 1010.

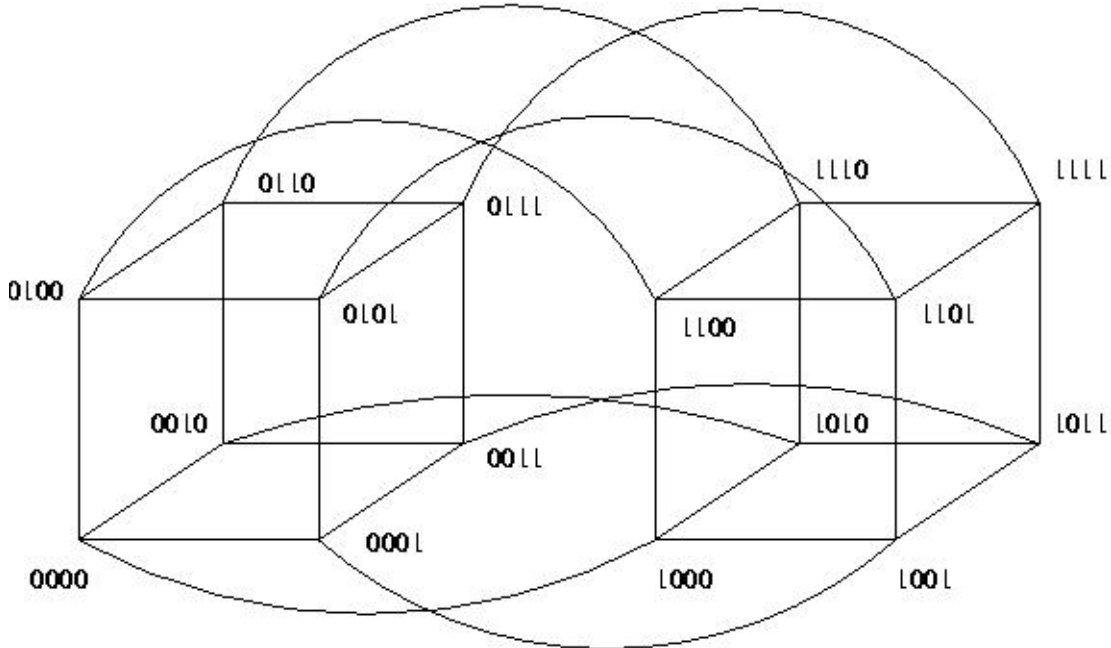


¹Note that we number the digits from right to left, with the rightmost digit being digit 0.

It is sometimes helpful to build up a cube from the lower-dimensional cases. To build a $(d+1)$ -dimensional cube from two d -dimensional cubes, just follow this recipe:

- (a) Take a d -dimensional cube and duplicate it. Call these two cubes subcube 0 and subcube 1.
- (b) For each pair of same-numbered PEs in the two subcubes, add a binary digit 0 to the front of the number for the PE in subcube 0, and add a 1 in the case of subcube 1. Add a link between them.

The following figure shows how a 4-cube can be constructed in this way from two 3-cubes:



Given a PE of number $(c_{d-1}, \dots, c_0)$ in a d -cube, we will discuss the i -cube to which this PE belongs, meaning all PEs whose first $d-i$ digits match this PE's.² Of all these PEs, the one whose last i digits are all 0s is called the **root** of this i -cube.

For the 4-cube and PE 1011 mentioned above, for instance, the 2-cube to which that PE belongs consists of 1000, 1001, 1010 and 1011—i.e. all PEs whose first two digits are 10—and the root is 1000.

Given a PE, we can split the i -cube to which it belongs into two $(i-1)$ -subcubes, one consisting of those PEs whose digit $i-1$ is 0 (to be called subcube 0), and the other consisting of those PEs whose digit $i-1$ is 1 (to be called subcube 1). Each

²Note that this is indeed an i -dimensional cube, because the last i digits are free to vary.

given PE in subcube 0 has as its **partner** the PE in subcube 1 whose digits match those of the given PE, except for digit $i-1$.

To illustrate this, again consider the 4-cube and the PE 1011. As an example, let us look at how the 3-cube it belongs to will split into two 2-cubes. The 3-cube to which 1011 belongs consists of 1000, 1001, 1010, 1011, 1100, 1101, 1110 and 1111. This 3-cube can be split into two 2-cubes, one being 1000, 1001, 1010 and 1011, and the other being 1100, 1101, 1110 and 1111. Then PE 1000 is partners with PE 1100, PE 1001 is partners with PE 1101, and so on.

Each link between two PEs is a dedicated connection, much preferable to the shared link we have when we run, say, MPI, on a collection of workstations on an Ethernet. On the other hand, if one PE needs to communicate with a non-neighbor PE, multiple links (as many as d of them) will need to be traversed. Thus the nature of the communications costs here is much different than for a network of workstations, and this must be borne in mind when developing programs.

7.3 Networks of Workstations (NOWs)

The idea here is simple: Take a bunch of commodity PCs and network them for use as parallel processing systems. They are of course individual machines, capable of the usual uniprocessor, nonparallel applications, but by networking them together and using message-passing software environments such as MPI, we can form very powerful parallel systems.

The networking does result in a significant loss of performance, but the price/performance ratio in NOW can be much superior in many applications to that of shared-memory or hypercube hardware of comparable number of CPUs.

7.3.1 The Network Is Literally the Weakest Link

Still, one factor which can be key to the success of a NOW is to use a fast network, both in terms of hardware and network protocol. Ordinary Ethernet and TCP/IP are fine for the applications envisioned by the original designers of the Internet, e.g. e-mail and file transfer, but they are slow in the NOW context.

A popular network for a NOW today is Infiniband (IB) (www.infinibandta.org). It features low latency, about 1.0-3.0 microseconds, high bandwidth, about 1.0-2.0 gigaBytes per second), and uses a low amount of the CPU's cycles, around 5-10%.

The basic building block of IB is a switch, with many inputs and outputs, similar in concept to Ω -net. You can build arbitrarily large and complex topologies from these switches.

A central point is that IB, as with other high-performance networks designed for NOWs, uses RDMA (Remote Direct Memory Access) read/write, which eliminates the extra copying of data between the application program's address space to that of the operating system.

IB has high performance and scalable³ implementations of distributed locks, semaphores, collective communication operations. An atomic operation takes about 3-5 microseconds.

IB implements true **multicast**, i.e. the simultaneous sending of messages to many nodes. Note carefully that even though MPI has its **MPI_Bcast()** function, it will send things out one at a time unless your network hardware is capable of multicast, and the MPI implementation you use is configured specifically for that hardware.

For information on network protocols, e.g. for example www.rdmaconsortium.org. A research paper evaluating a tuned implementation of MPI on IB is available at nowlab.cse.ohio-state.edu/publications/journal-papers/2004/liuj-ijpp04.pdf.

7.3.2 Other Issues

Increasingly today, the workstations themselves are multiprocessor machines, so a NOW really is a hybrid arrangement. They can be programmed either purely in a message-passing manner—e.g. running eight MPI processes on four dual-core machines—or in a mixed way, with a shared-memory approach being used within a workstation but message-passing used between them.

NOWs have become so popular that there are now “recipes” on how to build them for the specific purpose of parallel processing. The term **Beowulf** came to mean a NOW, usually with a fast network connecting them, used for parallel processing. The term *NOW* itself is no longer in use, replaced by *cluster*. Software packages such as ROCKS (<http://www.rockclusters.org/wordpress/>) have been developed to make it easy to set up and administer such systems.

7.4 Scatter/Gather Operations

Writing message-passing code is a lot of work, as the programmer must explicitly arrange for transfer of data. Contrast that, for instance, to shared-memory ma-

³The term *scalable* arises frequently in conversations on parallel processing. It means that this particular method of dealing with some aspect of parallel processing continues to work well as the system size increases. We say that the method *scales*.

chines, in which cache coherency transactions will cause data transfers, but which are not arranged by the programmer and not even seen by him/her.

In order to make coding on message-passing machines easier, higher-level systems have been devised. These basically operate in the **scatter/gather** paradigm, in which a “manager” node sends out chunks of work to the other nodes, serving as “workers,” and then collects and assembles the results sent back the workers.

MPI includes scatter/gather operations in its wide offering of functions, and they are used in many MPI applications. R’s **snow** package, which will be discussed in Section ??, is based entirely on scatter/gather, as is MapReduce, to be discussed below.

DISTRIBUTED PROGRAMMING WITH MPI

8.1 Introduction to MPI

8.2 Message Passing

8.2.1 Communication Routines

8.2.2 Collective Communications

8.3 Performance

INTRODUCTION TO MPI

MPI is the *de facto* standard for message-passing software.

9.1 Overview

9.1.1 History

Though (small) shared-memory machines have come down radically in price, to the point at which a dual-core PC is now commonplace in the home, historically shared-memory machines were available only to the “very rich”—large banks, national research labs and so on. This led to interest in message-passing machines.

The first “affordable” message-machine type was the Hypercube, developed by a physics professor at Cal Tech. It consisted of a number of **processing elements** (PEs) connected by fast serial I/O cards. This was in the range of university departmental research labs. It was later commercialized by Intel and NCube.

Later, the notion of **networks of workstations** (NOWs) became popular. Here the PEs were entirely independent PCs, connected via a standard network. This was refined a bit, by the use of more suitable network hardware and protocols, with the new term being **clusters**.

All of this necessitated the development of standardized software tools based on a message-passing paradigm. The first popular such tool was Parallel Virtual Machine (PVM). It still has its adherents today, but has largely been supplanted by the Message Passing Interface (MPI).

MPI itself later became MPI 2. Our document here is intended mainly for the original.

9.1.2 Structure and Execution

MPI is merely a set of Application Programmer Interfaces (APIs), called from user programs written in C, C++ and other languages. It has many implementations, with some being open source and generic, while others are proprietary and fine-tuned for specific commercial hardware.

Suppose we have written an MPI program **x**, and will run it on four machines in a cluster. Each machine will be running its own copy of **x**. Official MPI terminology refers to this as four **processes**. Now that multicore machines are commonplace, one might indeed run two or more cooperating MPI processes—where now we use the term *processes* in the real OS sense—on the same multicore machine. In this document, we will tend to refer to the various MPI processes as **nodes**, with an eye to the cluster setting.

Though the nodes are all running the same program, they will likely be working on different parts of the program's data. This is called the Single Program Multiple Data (SPMD) model. This is the typical approach, but there could be different programs running on different nodes. Most of the APIs involve a node sending information to, or receiving information from, other nodes.

9.1.3 Implementations

Two of the most popular implementations of MPI are MPICH and LAM. MPICH offers more tailoring to various networks and other platforms, while LAM runs on networks. Introductions to MPICH and LAM can be found, for example, at <http://heather.cs.ucdavis.edu/~matloff/MPI/NotesMPICH.NM.html> and <http://heather.cs.ucdavis.edu/~matloff/MPI/NotesLAM.NM.html>, respectively.

LAM is no longer being developed, and has been replaced by Open MPI (not to be confused with OpenMP). Personally, I still prefer the simplicity of LAM. It is still being maintained.

Note carefully: If your machine has more than one MPI implementation, make absolutely sure one is not interfering with the other. Make sure all execution and library paths all include one and only one implementation at a time.

9.1.4 Performance Issues

Mere usage of a parallel language on a parallel platform does not guarantee a performance improvement over a serial version of your program. The central issue here is the overhead involved in internode communication.

Infiniband, one of the fastest cluster networks commercially available, has a **latency** of about 1.0-3.0 microseconds, meaning that it takes the first bit of a packet that long to get from one node on an Infiniband switch to another. Comparing that to the nanosecond time scale of CPU speeds, one can see that the communications overhead can destroy a program's performance. And Ethernet is quite a bit slower than Infiniband.

Latency is quite different from **bandwidth**, which is the number of bits sent per second. Say the latency is 1.0 microsecond and the bandwidth is 1 gigabit, i.e. 1000000000 bits per second or 1000 bits per microsecond. Say the message is 2000 bits long. Then the first bit of the message arrives after 1 microsecond, and the last bit arrives after an additional 2 microseconds. In other words, the message is does not arrive fully at the destination until 3 microseconds after it is sent.

In the same setting, say bandwidth is 10 gigabits. Now the message would need 1.2 seconds to arrive fully, in spite of a 10-fold increase in bandwidth. So latency is a major problem even if the bandwidth is high.

For this reason, the MPI applications that run well on networks tend to be of the “embarrassingly parallel” type, with very little communication between the processes.

Of course, if your platform is a shared-memory multiprocessor (especially a multicore one, where communication between cores is particularly fast) and you are running all your MPI processes on that machine, the problem is less severe. In fact, some implementations of MPI communicate directly through shared memory in that case, rather than using the TCP/IP or other network protocol.

9.2 Review of Earlier Example

Though the presentation in this chapter is self-contained, you may wish to look first at the somewhat simpler example in Section ??, a pipelined prime number finder.

9.3 Example: Dijkstra Algorithm

9.3.1 The Algorithm

The code implements the Dijkstra algorithm for finding the shortest paths in an undirected graph. Pseudocode for the algorithm is

```

1  Done = {0}
2  NonDone = {1,2,...,N-1}
3  for J = 1 to N-1 Dist[J] = infinity
4  Dist[0] = 0
5  for Step = 1 to N-1
6      find J such that Dist[J] is min among all J in NonDone
7      transfer J from NonDone to Done
8      NewDone = J
9      for K = 1 to N-1
10         if K is in NonDone
11             Dist[K] = min(Dist[K],Dist[NewDone]+G[NewDone,K])

```

At each iteration, the algorithm finds the closest vertex J to 0 among all those not yet processed, and then updates the list of minimum distances to each vertex from 0 by considering paths that go through J. Two obvious potential candidate part of the algorithm for parallelization are the “find J” and “for K” lines, and the above OpenMP code takes this approach.

9.3.2 The MPI Code

```

1  // Dijkstra.c
2
3  // MPI example program: Dijkstra shortest-path finder in a
4  // bidirectional graph; finds the shortest path from vertex 0 to all
5  // others
6
7  // command line arguments: nv print dbg
8
9  // where: nv is the size of the graph; print is 1 if graph and min
10 // distances are to be printed out, 0 otherwise; and dbg is 1 or 0, 1
11 // for debug
12
13 // node 0 will both participate in the computation and serve as a
14 // "manager"
15
16 #include <stdio.h>
17 #include <mpi.h>
18
19 #define MYMIN_MSG 0
20 #define OVRLMIN_MSG 1
21 #define COLLECT_MSG 2
22
23 // global variables (but of course not shared across nodes)
24
25 int nv, // number of vertices
26     *notdone, // vertices not checked yet
27     nnodes, // number of MPI nodes in the computation
28     chunk, // number of vertices handled by each node
29     startv,endv, // start, end vertices for this node
30     me, // my node number
31     dbg;
32 unsigned largeint, // max possible unsigned int
33     mymin[2], // mymin[0] is min for my chunk,
34              // mymin[1] is vertex which achieves that min
35     othermin[2], // othermin[0] is min over the other chunks
36                // (used by node 0 only)
37                // othermin[1] is vertex which achieves that min

```

```

38         overallmin[2], // overallmin[0] is current min over all nodes,
39                        // overallmin[1] is vertex which achieves that min
40         *ohd, // 1-hop distances between vertices; "ohd[i][j]" is
41              // ohd[i*nv+j]
42         *mind; // min distances found so far
43
44     double T1,T2; // start and finish times
45
46     void init(int ac, char **av)
47     { int i,j,tmp; unsigned u;
48       nv = atoi(av[1]);
49       dbg = atoi(av[3]);
50       MPI_Init(&ac,&av);
51       MPI_Comm_size(MPI_COMM_WORLD,&nnodes);
52       MPI_Comm_rank(MPI_COMM_WORLD,&me);
53       chunk = nv/nnodes;
54       startv = me * chunk;
55       endv = startv + chunk - 1;
56       u = -1;
57       largeint = u >> 1;
58       ohd = malloc(nv*nv*sizeof(int));
59       mind = malloc(nv*sizeof(int));
60       notdone = malloc(nv*sizeof(int));
61       // random graph
62       // note that this will be generated at all nodes; could generate just
63       // at node 0 and then send to others, but faster this way
64       srand(9999);
65       for (i = 0; i < nv; i++)
66         for (j = i; j < nv; j++) {
67           if (j == i) ohd[i*nv+i] = 0;
68           else {
69             ohd[nv*i+j] = rand() % 20;
70             ohd[nv*j+i] = ohd[nv*i+j];
71           }
72         }
73       for (i = 0; i < nv; i++) {
74         notdone[i] = 1;
75         mind[i] = largeint;
76       }
77       mind[0] = 0;
78       while (dbg) ; // stalling so can attach debugger
79     }
80
81     // finds closest to 0 among notdone, among startv through endv
82     void findmymin()
83     { int i;
84       mymin[0] = largeint;
85       for (i = startv; i <= endv; i++)
86         if (notdone[i] && mind[i] < mymin[0]) {
87           mymin[0] = mind[i];
88           mymin[1] = i;
89         }
90     }
91
92     void findoverallmin()
93     { int i;
94       MPI_Status status; // describes result of MPI_Recv() call
95       // nodes other than 0 report their mins to node 0, which receives
96       // them and updates its value for the global min
97       if (me > 0)
98         MPI_Send(mymin,2,MPI_INT,0,MYMIN_MSG,MPI_COMM_WORLD);

```

```

99     else {
100         // check my own first
101         overallmin[0] = mymin[0];
102         overallmin[1] = mymin[1];
103         // check the others
104         for (i = 1; i < nnodes; i++) {
105             MPI_Recv(othermin,2,MPI_INT,i,MYMIN_MSG,MPI_COMM_WORLD,&status);
106             if (othermin[0] < overallmin[0]) {
107                 overallmin[0] = othermin[0];
108                 overallmin[1] = othermin[1];
109             }
110         }
111     }
112 }
113
114 void updatemymind() // update my mind segment
115 { // for each i in [startv,endv], ask whether a shorter path to i
116   // exists, through mv
117   int i, mv = overallmin[1];
118   unsigned md = overallmin[0];
119   for (i = startv; i <= endv; i++)
120       if (md + ohd[mv*nv+i] < mind[i])
121           mind[i] = md + ohd[mv*nv+i];
122 }
123
124 void disseminateoverallmin()
125 { int i;
126   MPI_Status status;
127   if (me == 0)
128       for (i = 1; i < nnodes; i++)
129           MPI_Send(overallmin,2,MPI_INT,i,OVRLMIN_MSG,MPI_COMM_WORLD);
130   else
131       MPI_Recv(overallmin,2,MPI_INT,0,OVRLMIN_MSG,MPI_COMM_WORLD,&status);
132 }
133
134 void updateallmind() // collects all the mind segments at node 0
135 { int i;
136   MPI_Status status;
137   if (me > 0)
138       MPI_Send(mind+startv,chunk,MPI_INT,0,COLLECT_MSG,MPI_COMM_WORLD);
139   else
140       for (i = 1; i < nnodes; i++)
141           MPI_Recv(mind+i*chunk,chunk,MPI_INT,i,COLLECT_MSG,MPI_COMM_WORLD,
142                 &status);
143 }
144
145 void printmind() // partly for debugging (call from GDB)
146 { int i;
147   printf("minimum distances:\n");
148   for (i = 1; i < nv; i++)
149       printf("%u\n",mind[i]);
150 }
151
152 void dowork()
153 { int step, // index for loop of nv steps
154   i;
155   if (me == 0) T1 = MPI_Wtime();
156   for (step = 0; step < nv; step++) {
157       findmymin();
158       findoverallmin();
159       disseminateoverallmin();

```

```

160         // mark new vertex as done
161         notdone[overallmin[1]] = 0;
162         updatemymind(startv,endv);
163     }
164     updateallmind();
165     T2 = MPI_Wtime();
166 }
167
168 int main(int ac, char **av)
169 { int i,j,print;
170   init(ac,av);
171   dowork();
172   print = atoi(av[2]);
173   if (print && me == 0) {
174     printf("graph weights:\n");
175     for (i = 0; i < nv; i++) {
176       for (j = 0; j < nv; j++)
177         printf("%u ",ohd[nv*i+j]);
178       printf("\n");
179     }
180     printmind();
181   }
182   if (me == 0) printf("time at node 0: %f\n", (float)(T2-T1));
183   MPI_Finalize();
184 }
185

```

The various MPI functions will be explained in the next section.

9.3.3 Introduction to MPI APIs

MPI_Init() and MPI_Finalize()

These are required for starting and ending execution of an MPI program. Their actions may be implementation-dependent. For instance, if our platform is an Ethernet-based cluster, **MPI_Init()** will probably set up the TCP/IP sockets via which the various nodes communicate with each other. On an Infiniband-based cluster, connections in the special Infiniband network protocol will be established. On a shared-memory multiprocessor, an implementation of MPI that is tailored to that platform would take very different actions.

MPI_Comm_size() and MPI_Comm_rank()

In our function **init()** above, note the calls

```

MPI_Comm_size(MPI_COMM_WORLD,&nnodes);
MPI_Comm_rank(MPI_COMM_WORLD,&me);

```

The first call determines how many nodes are participating in our computation, placing the result in our variable **nnodes**. Here **MPI_COMM_WORLD** is our node group, termed a **communicator** in MPI parlance. MPI allows the

programmer to subdivide the nodes into groups, to facilitate performance and clarity of code. Note that for some operations, such as barriers, the only way to apply the operation to a proper subset of all nodes is to form a group. The totality of all groups is denoted by **MPI_COMM_WORLD**. In our program here, we are not subdividing into groups.

The second call determines this node's ID number, called its **rank**, within its group. As mentioned earlier, even though the nodes are all running the same program, they are typically working on different parts of the program's data. So, the program needs to be able to sense which node it is running on, so as to access the appropriate data. Here we record that information in our variable **me**.

MPI_Send()

To see how MPI's basic send function works, consider our line above,

```
MPI_Send(mymin,2,MPI_INT,0,MYMIN_MSG,MPI_COMM_WORLD);
```

Let's look at the arguments:

mymin: We are sending a set of bytes. This argument states the address at which these bytes begin.

2, MPI_INT: This says that our set of bytes to be sent consists of 2 objects of type **MPI_INT**. That means 8 bytes on 32-bit machines, so why not just collapse these two arguments to one, namely the number 8? Why did the designers of MPI bother to define data types? The answer is that we want to be able to run MPI on a heterogeneous set of machines, with MPI serving as the "broker" between them in case different architectures among those machines handle data differently.

First of all, there is the issue of **endianness**. Intel machines, for instance, are **little-endian**, which means that the least significant byte of a memory word has the smallest address among bytes of the word. Sun SPARC chips, on the other hand, are **big-endian**, with the opposite storage scheme. If our set of nodes included machines of both types, straight transmission of sequences of 8 bytes might mean that some of the machines literally receive the data backwards!

Secondly, these days 64-bit machines are becoming more and more common. Again, if our set of nodes were to include both 32-bit and 64-bit words, some major problems would occur if no conversion were done.

0: We are sending to node 0.

MYMIN_MSG: This is the message type, programmer-defined in our line

```
#define MYMIN_MSG 0
```

Receive calls, described in the next section, can ask to receive only messages of a certain type.

_COMM_WORLD: This is the node group to which the message is to be sent. Above, where we said we are sending to node 0, we technically should say we are sending to node 0 within the group **MPI_COMM_WORLD**.

MPI_Recv()

Let's now look at the arguments for a basic receive:

```
MPI_Recv(othermin,2,MPI_INT,i,MYMIN_MSG,MPI_COMM_WORLD,&status);
```

othermin: The received message is to be placed at our location **othermin**.

2,MPI_INT: Two objects of **MPI_INT** type are to be received.

i: Receive only messages from node **i**. If we did not care what node we received a message from, we could specify the value **MPI_ANY_SOURCE**.

MYMIN_MSG: Receive only messages of type **MYMIN_MSG**. If we did not care what type of message we received, we would specify the value **MPI_ANY_TAG**.

_COMM_WORLD: Group name.

status: Recall our line

```
MPI_Status status; // describes result of MPI_Recv() call
```

The type is an MPI **struct** containing information about the received message. Its primary fields of interest are **MPI_SOURCE**, which contains the identity of the sending node, and **MPI_TAG**, which contains the message type. These would be useful if the receive had been done with **MPI_ANY_SOURCE** or **MPI_ANY_TAG**; the status argument would then tell us which node sent the message and what type the message was.

9.4 Example: Removing 0s from an Array

```
1 #include <mpi.h>
2 #include <stdlib.h>
```

```

3
4 #define MAX_N 100000
5 #define MAX_NPROCS 100
6 #define DATA_MSG 0
7 #define NEWDATA_MSG 1
8
9 int nnodes, // number of MPI processes
10     n, // size of original array
11     me, // my MPI ID
12     has0s[MAX_N], // original data
13     no0s[MAX_N], // 0-free data
14     nno0s; // number of non-0 elements
15
16 int debug;
17
18 init(int argc, char **argv)
19 {
20     int i;
21     MPI_Init(&argc,&argv);
22     MPI_Comm_size(MPI_COMM_WORLD,&nnodes);
23     MPI_Comm_rank(MPI_COMM_WORLD,&me);
24     n = atoi(argv[1]);
25     if (me == 0) {
26         for (i = 0; i < n; i++)
27             has0s[i] = rand() % 4;
28     } else {
29         debug = atoi(argv[2]);
30         while (debug) ;
31     }
32 }
33
34 void managernode()
35 {
36     MPI_Status status;
37     int i;
38     int lenchunk;
39     lenchunk = n / (nnodes-1); // assumed divides evenly
40     for (i = 1; i < nnodes; i++) {
41         MPI_Send(has0s+(i-1)*lenchunk, lenchunk,
42                 MPI_INT, i, DATA_MSG, MPI_COMM_WORLD);
43     }

```

```

44     int k = 0;
45     for (i = 1; i < nnodes; i++) {
46         MPI_Recv(no0s+k,MAX_N,
47             MPI_INT,i,NEWDATA_MSG,MPI_COMM_WORLD,&status);
48         MPI_Get_count(&status,MPI_INT,&lchunk);
49         k += lchunk;
50     }
51     nno0s = k;
52 }
53
54 void remov0s(int *oldx, int n, int *newx, int *nnewx)
55 { int i,count = 0;
56   for (i = 0; i < n; i++)
57       if (oldx[i] != 0) newx[count++] = oldx[i];
58   *nnewx = count;
59 }
60
61 void workernode()
62 {
63     int lchunk;
64     MPI_Status status;
65     MPI_Recv(has0s,MAX_N,
66         MPI_INT,0,DATA_MSG,MPI_COMM_WORLD,&status);
67     MPI_Get_count(&status,MPI_INT,&lchunk);
68     remov0s(has0s,lchunk,no0s,&nno0s);
69     MPI_Send(no0s,nno0s,
70         MPI_INT,0,NEWDATA_MSG,MPI_COMM_WORLD);
71 }
72
73 int main(int argc,char **argv)
74 {
75     int i;
76     init(argc,argv);
77     if (me == 0 && n < 25) {
78         for (i = 0; i < n; i++) printf("%d ",has0s[i]);
79         printf("\n");
80     }
81     if (me == 0) managernode();
82     else workernode();
83     if (me == 0 && n < 25) {
84         for (i = 0; i < n; i++) printf("%d ",no0s[i]);

```

```
85     printf("\n");
86 }
87 MPI_Finalize();
88 }
```

9.5 Debugging MPI Code

If you are using GDB—either directly, or via an IDE such as Eclipse or Netbeans—the trick with MPI is to **attach** GDB to your running MPI processes.

Set up code like that we’ve seen in our examples here:

```
1 while (dbg) ;
```

This deliberately sets up an infinite loop of **dbg** is nonzero, for reasons to be discussed below.

For instance, suppose I’m running an MPI program **a.out**, on machines A, B and C. I would start the processes as usual, and have three terminal windows open. I’d log in to machine A, find the process number for **a.out**, using for example a command like **ps ax** on Unix-family systems, then attach GDB to that process. Say the process number is 88888. I’d attach by running the command

```
% gdb a.out 88888
```

That would start GDB, in the midst of my already-running process, thus stuck in the infinite loop seen above. I hit ctrl-c to interrupt it, which gives me the GDB prompt, (gdb). I then type

```
(gdb) set var dbg = 0
```

which means when I next hit the **c** command in GDB, the program will proceed, not stuck in the loop anymore. But first I set my breakpoints.

9.6 Collective Communications

MPI features a number of **collective communication** capabilities, a number of which are used in the following refinement of our Dijkstra program:

9.6.1 Example: Refined Dijkstra Code

```

1 // Dijkstra.coll1.c
2
3 // MPI example program: Dijkstra shortest-path finder in a
4 // bidirectional graph; finds the shortest path from vertex 0 to all
5 // others; this version uses collective communication
6
7 // command line arguments: nv print dbg
8
9 // where: nv is the size of the graph; print is 1 if graph and min
10 // distances are to be printed out, 0 otherwise; and dbg is 1 or 0, 1
11 // for debug
12
13 // node 0 will both participate in the computation and serve as a
14 // "manager"
15
16 #include <stdio.h>
17 #include <mpi.h>
18
19 // global variables (but of course not shared across nodes)
20
21 int nv, // number of vertices
22     *notdone, // vertices not checked yet
23     nnodes, // number of MPI nodes in the computation
24     chunk, // number of vertices handled by each node
25     startv, endv, // start, end vertices for this node
26     me, // my node number
27     dbg;
28 unsigned largeint, // max possible unsigned int
29     mymin[2], // mymin[0] is min for my chunk,
30               // mymin[1] is vertex which achieves that min
31     overallmin[2], // overallmin[0] is current min over all nodes,
32                   // overallmin[1] is vertex which achieves that min
33     *ohd, // 1-hop distances between vertices; "ohd[i][j]" is
34           // ohd[i*nv+j]
35     *mind; // min distances found so far
36
37 double T1,T2; // start and finish times
38
39 void init(int ac, char **av)
40 { int i,j,tmp; unsigned u;
41   nv = atoi(av[1]);
42   dbg = atoi(av[3]);
43   MPI_Init(&ac,&av);
44   MPI_Comm_size(MPI_COMM_WORLD,&nnodes);
45   MPI_Comm_rank(MPI_COMM_WORLD,&me);
46   chunk = nv/nnodes;
47   startv = me * chunk;
48   endv = startv + chunk - 1;
49   u = -1;
50   largeint = u >> 1;
51   ohd = malloc(nv*nv*sizeof(int));
52   mind = malloc(nv*sizeof(int));
53   notdone = malloc(nv*sizeof(int));
54   // random graph
55   // note that this will be generated at all nodes; could generate just
56   // at node 0 and then send to others, but faster this way
57   srand(9999);
58   for (i = 0; i < nv; i++)
59     for (j = i; j < nv; j++) {
60       if (j == i) ohd[i*nv+i] = 0;
61       else {

```

```

62         ohd[nv*i+j] = rand() % 20;
63         ohd[nv*j+i] = ohd[nv*i+j];
64     }
65 }
66 for (i = 0; i < nv; i++) {
67     notdone[i] = 1;
68     mind[i] = largeint;
69 }
70 mind[0] = 0;
71 while (dbg) ; // stalling so can attach debugger
72 }
73
74 // finds closest to 0 among notdone, among startv through endv
75 void findmymin()
76 { int i;
77   mymin[0] = largeint;
78   for (i = startv; i <= endv; i++)
79       if (notdone[i] && mind[i] < mymin[0]) {
80           mymin[0] = mind[i];
81           mymin[1] = i;
82       }
83 }
84
85 void updatemymind() // update my mind segment
86 { // for each i in [startv,endv], ask whether a shorter path to i
87   // exists, through mv
88   int i, mv = overallmin[1];
89   unsigned md = overallmin[0];
90   for (i = startv; i <= endv; i++)
91       if (md + ohd[mv*nv+i] < mind[i])
92           mind[i] = md + ohd[mv*nv+i];
93 }
94
95 void printmind() // partly for debugging (call from GDB)
96 { int i;
97   printf("minimum distances:\n");
98   for (i = 1; i < nv; i++)
99       printf("%u\n",mind[i]);
100 }
101
102 void dowork()
103 { int step, // index for loop of nv steps
104   i;
105   if (me == 0) T1 = MPI_Wtime();
106   for (step = 0; step < nv; step++) {
107       findmymin();
108       MPI_Reduce(mymind,overallmin,1,MPI_2INT,MPI_MINLOC,0,MPI_COMM_WORLD);
109       MPI_Bcast(overallmin,1,MPI_2INT,0,MPI_COMM_WORLD);
110       // mark new vertex as done
111       notdone[overallmin[1]] = 0;
112       updatemymind(startv,endv);
113   }
114   // now need to collect all the mind values from other nodes to node 0
115   MPI_Gather(mind+startv,chunk,MPI_INT,mind,chunk,MPI_INT,0,MPI_COMM_WORLD);
116   T2 = MPI_Wtime();
117 }
118
119 int main(int ac, char **av)
120 { int i,j,print;
121   init(ac,av);
122   dowork();

```

```

123     print = atoi(av[2]);
124     if (print && me == 0) {
125         printf("graph weights:\n");
126         for (i = 0; i < nv; i++) {
127             for (j = 0; j < nv; j++)
128                 printf("%u ",ohd[nv*i+j]);
129             printf("\n");
130         }
131         printmind();
132     }
133     if (me == 0) printf("time at node 0: %f\n", (float)(T2-T1));
134     MPI_Finalize();
135 }

```

The new calls will be explained in the next section.

9.6.2 MPI_Bcast()

In our original Dijkstra example, we had a loop

```

for (i = 1; i < nnodes; i++)
    MPI_Send(overallmin,2,MPI_INT,i,OVRLMIN_MSG,MPI_COMM_WORLD);

```

in which node 0 sends to all other nodes. We can replace this by

```

MPI_Bcast(overallmin,2,MPI_INT,0,MPI_COMM_WORLD);

```

In English, this call would say,

At this point all nodes participate in a broadcast operation, in which node 0 sends 2 objects of type **MPI_INT** to each node (including itself). The source of the data will be located at address **overallmin** at node 0, and the other nodes will receive the data at a location of that name.

Note my word “participate” above. The name of the function is “broadcast,” which makes it sound like only node 0 executes this line of code, which is not the case; all the nodes in the group (in this case that means all nodes in our entire computation) execute this line. The only difference is the action; most nodes participate by receiving, while node 0 participates by sending.

Actually, this call to **MPI_Bcast()** is doing more than replacing the loop, since the latter had been part of an if-then-else that checked whether the given process had rank 0 or not.

Why might this be preferable to using an explicit loop? First, it would obviously be much clearer. That makes the program easier to write, easier to debug, and easier for others (and ourselves, later) to read.

But even more importantly, using the broadcast may improve performance. We may, for instance, be using an implementation of MPI which is tailored to the platform on which we are running MPI. If for instance we are running on a network designed for parallel computing, such as Myrinet or Infiniband, an optimized broadcast may achieve a much higher performance level than would simply a loop with individual send calls. On a shared-memory multiprocessor system, special machine instructions specific to that platform's architecture can be exploited, as for instance IBM has done for its shared-memory machines. Even on an ordinary Ethernet, one could exploit Ethernet's own broadcast mechanism, as had been done for PVM, a system like MPI (G. Davies and N. Matloff, Network-Specific Performance Enhancements for PVM, *Proceedings of the Fourth IEEE International Symposium on High-Performance Distributed Computing*, 1995, 205-210; N. Matloff, Analysis of a Programmed Backoff Method for Parallel Processing on Ethernets, in *Network-Based Parallel Computing*).

9.6.3 MPI_Reduce()/MPI_Allreduce()

Look at our call

```
MPI_Reduce(mymin,overallmin,1,MPI_2INT,MPI_MINLOC,0,MPI_COMM_WORLD);
```

above. In English, this would say,

At this point all nodes in this group participate in a “reduce” operation. The type of reduce operation is **MPI_MINLOC**, which means that the minimum value among the nodes will be computed, and the index attaining that minimum will be recorded as well. Each node contributes a value to be checked, and an associated index, from a location **mymin** in their programs; the type of the pair is **MPI_2INT**. The overall min value/index will be computed by combining all of these values at node 0, where they will be placed at a location **overallmin**.

MPI also includes a function **MPI_Allreduce()**, which does the same operation, except that instead of just depositing the result at one node, it does so at all nodes. So for instance our code above,

```
MPI_Reduce(mymin,overallmin,1,MPI_2INT,MPI_MINLOC,0,MPI_COMM_WORLD);
MPI_Bcast(overallmin,1,MPI_2INT,0,MPI_COMM_WORLD);
```

could be replaced by

```
MPI_Allreduce(mymin,overallmin,1,MPI_2INT,MPI_MINLOC,MPI_COMM_WORLD);
```

Again, these can be optimized for particular platforms.

Here is a table of MPI reduce operations:

MPI_MAX	max
MPI_MIN	min
MPI_SUM	sum
MPI_PROD	product
MPI_LAND	wordwise boolean and
MPI_LOR	wordwise boolean or
MPI_LXOR	wordwise exclusive or
MPI_BAND	bitwise boolean and
MPI_BOR	bitwise boolean or
MPI_BXOR	bitwise exclusive or
MPI_MAXLOC	max value and location
MPI_MINLOC	min value and location

9.6.4 MPI_Gather()/MPI_Allgather()

A classical approach to parallel computation is to first break the data for the application into chunks, then have each node work on its chunk, and then gather all the processed chunks together at some node. The MPI function **MPI_Gather()** does this.

In our program above, look at the line

```
MPI_Gather(mind+startv,chunk,MPI_INT,mind,chunk,MPI_INT,0,MPI_COMM_WORLD);
```

In English, this says,

At this point all nodes participate in a gather operation, in which each node (including Node 0) contributes **chunk** number of MPI integers, from a location **mind+startv** in that node's program. Node 0 then receives **chunk** items sent from each node, stringing everything together in node order and depositing it all at **mind** in the program running at Node 0.

(Yes, the fifth argument is redundant with the second; same for the third and sixth.)

There is also **MPI_Allgather()**, which places the result at all nodes, not just one. Its call form is the same as **MPI_Gather()**, but with one fewer argument (since the identity of "the" gathering node is no longer meaningful):

```
int MPI_Allgather(srcbuf, srccount, srctype, destbuf, destcount, destt
```

9.6.5 The MPI_Scatter()

This is the opposite of **MPI_Gather()**, i.e. it breaks long data into chunks which it parcels out to individual nodes. For example, in the code in the next section, the call

```
MPI_Scatter(oh, lenchunk, MPI_INT, ohchunk, lenchunk, MPI_INT, 0,
            MPI_COMM_WORLD);
```

means

Node 0 will break up the array **oh** of type **MPI_INT** into chunks of length **lenchunk**, sending the i^{th} chunk to Node i , where **lenchunk** items will be deposited at **ohchunk**.

9.6.6 Example: Count the Number of Edges in a Directed Graph

Below is MPI code to count the number of edges in a directed graph. (“Directed” means that a link from i to j does not necessarily imply one from j to i .)

In the context here, **me** is the node’s rank; **nv** is the number of vertices; **oh** is the one-hop distance matrix; and **nnodes** is the number of MPI processes. At the beginning only the process of rank 0 has a copy of **oh**, but it sends that matrix out in chunks to the other nodes, each of which stores its chunk in an array **ohchunk**.

```
1 lenchunk = nv / nnodes;
2 MPI_Scatter(oh, lenchunk, MPI_INT, ohchunk, lenchunk, MPI_INT, 0,
3     MPI_COMM_WORLD);
4 mycount = 0;
5 for (i = 0; i < nv*nv/nnodes)
6     if (ohchunk[i] != 0) mycount++;
7 MPI_Reduce(&mycount, &numedge, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
8 if (me == 0) printf("there are %d edges\n", numedge);
```

9.6.7 Example: Cumulative Sums

Here we find cumulative sums. For instance, if the original array is (3,1,2,0,3,0,1,2), then it is changed to (3,4,6,6,9,9,10,12). (This topic is pursued in depth in Chapter ??.)

```
1 // finds cumulative sums in the array x
2
3 #include <mpi.h>
```

```

4 #include <stdlib.h>
5
6 #define MAX_N 10000000
7 #define MAX_NODES 10
8
9 int nnodes, // number of MPI processes
10     n, // size of x
11     me, // MPI rank of this node
12     // full data for node 0, part for the rest
13     x[MAX_N],
14     csums[MAX_N], // cumulative sums for this node
15     maxvals[MAX_NODES]; // the max values at the various nodes
16
17 int debug;
18
19 init(int argc, char **argv)
20 {
21     int i;
22     MPI_Init(&argc,&argv);
23     MPI_Comm_size(MPI_COMM_WORLD,&nnodes);
24     MPI_Comm_rank(MPI_COMM_WORLD,&me);
25     n = atoi(argv[1]);
26     // test data
27     if (me == 0) {
28         for (i = 0; i < n; i++)
29             x[i] = rand() % 32;
30     }
31     debug = atoi(argv[2]);
32     while (debug) ;
33 }
34
35 void cumulsums()
36 {
37     MPI_Status status;
38     int i, lenchunk, sum, node;
39     lenchunk = n / nnodes; // assumed to divide evenly
40     // note that node 0 will participate in the computation too
41     MPI_Scatter(x, lenchunk, MPI_INT, x, lenchunk, MPI_INT,
42         0, MPI_COMM_WORLD);
43     sum = 0;
44     for (i = 0; i < lenchunk; i++) {

```

```

45         csums[i] = sum + x[i];
46         sum += x[i];
47     }
48     MPI_Gather(&csums[lenchunk-1], 1, MPI_INT,
49         maxvals, 1, MPI_INT, 0, MPI_COMM_WORLD);
50     MPI_Bcast(maxvals, nnodes, MPI_INT, 0, MPI_COMM_WORLD);
51     if (me > 0) {
52         sum = 0;
53         for (node = 0; node < me; node++) {
54             sum += maxvals[node];
55         }
56         for (i = 0; i < lenchunk; i++)
57             csums[i] += sum;
58     }
59     MPI_Gather(csums, lenchunk, MPI_INT, csums, lenchunk, MPI_INT,
60         0, MPI_COMM_WORLD);
61 }
62
63 int main(int argc, char **argv)
64 {
65     int i;
66     init(argc, argv);
67     if (me == 0 && n < 25) {
68         for (i = 0; i < n; i++) printf("%d ", x[i]);
69         printf("\n");
70     }
71     cumulsums();
72     if (me == 0 && n < 25) {
73         for (i = 0; i < n; i++) printf("%d ", csums[i]);
74         printf("\n");
75     }
76     MPI_Finalize();
77 }

```

9.6.8 Example: an MPI Solution to the Mutual Outlinks Problem

Consider the example of Section ?? . We have a network graph of some kind, such as Web links. For any two vertices, say any two Web sites, we might be interested in mutual outlinks, i.e. outbound links that are common to two Web sites.

The MPI code below finds the mean number of mutual outlinks, among all pairs of vertices in a graph.

```
1 // MPI solution to the mutual outlinks problem
2
3 // adjacency matrix m is global at each node, broadcast from node 0
4
5 // assumes m is nxn, and number of nodes is < n
6
7 // for each node i, check all possible pairing nodes j > i; the various
8 // nodes work on values of i in a Round Robin fashion, with node k
9 // handling all i for which i mod nnodes = k
10
11 #include <mpi.h>
12 #include <stdlib.h>
13
14 #define MAXLENGTH 10000000
15
16 int nnodes, // number of MPI processes
17     n, // size of x
18     me, // MPI rank of this node
19     m[MAXLENGTH], // adjacency matrix
20     grandtot; // grand total of all counts of mutuality
21
22 // get adjacency matrix, in this case just by simulation
23 void getm()
24 { int i;
25   for (i = 0; i < n*n; i++)
26     m[i] = rand() % 2;
27 }
28
29 init(int argc, char **argv)
30 {
31   int i;
32   MPI_Init(&argc,&argv);
33   MPI_Comm_size(MPI_COMM_WORLD,&nnodes);
34   MPI_Comm_rank(MPI_COMM_WORLD,&me);
35   n = atoi(argv[1]);
36   if (me == 0) {
37     getm(); // get the data (app-specific)
38   }
```

```

39 }
40
41 void mutlinks()
42 {
43     int i,j,k,tot;
44     MPI_Bcast(m,n*n,MPI_INT,0,MPI_COMM_WORLD);
45     tot = 0;
46     for (i = me; i < n-1; i += nnodes) {
47         for (j = i+1; j < n; j++) {
48             for (k = 0; k < n; k++)
49                 tot += m[twod2oned(n,i,k)] * m[twod2oned(n,j,k)];
50         }
51     }
52     MPI_Reduce(&tot,&grandtot,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);
53 }
54
55 // convert 2-D subscript to 1-D
56 int twod2oned(n,i,j)
57 { return n * i + j; }
58
59 int main(int argc,char **argv)
60 { int i,j;
61     init(argc,argv);
62     if (me == 0 && n < 5) { // check test input
63         for (i = 0; i < n; i++) {
64             for (j = 0; j < n; j++) printf("%d ",m[twod2oned(n,i,j)]);
65             printf("\n");
66         }
67     }
68     mutlinks();
69     if (me == 0) printf("%f\n",((float) grandtot)/(n*(n-1)/2));
70     MPI_Finalize();
71 }

```

9.6.9 The MPI_Barrier()

This implements a barrier for a given communicator. The name of the communicator is the sole argument for the function.

Explicit barriers are less common in message-passing programs than in the shared-memory world.

9.6.10 Creating Communicators

Again, a communicator is a subset (either proper or improper) of all of our nodes. MPI includes a number of functions for use in creating communicators. Some set up a virtual “topology” among the nodes.

For instance, many physics problems consist of solving differential equations in two- or three-dimensional space, via approximation on a grid of points. In two dimensions, groups may consist of rows in the grid.

Here’s how we might divide an MPI run into two groups (assumes an even number of MPI processes to begin with):

```
MPI_Comm_size(MPI_COMM_WORLD,&nnodes);
MPI_Comm_rank(MPI_COMM_WORLD,&me);
...
// declare variables to bind to groups
MPI_Group worldgroup, subgroup;
// declare variable to bind to a communicator
MPI_Comm subcomm;
...
int i,startrank,nn2 = nnodes/2;
int *subranks = malloc(nn2*sizeof(int));
if (me < nn2) start = 0;
else start = nn2;
for (i = 0; i < nn2; i++)
    subranks[i] = i + start;
// bind the world to a group variable
MPI_Comm_group(MPI_COMM_WORLD, &worldgroup);
// take worldgroup the nn2 ranks in "subranks" and form group
// "subgroup" from them
MPI_Group_incl(worldgroup, nn2, subranks, subgroup);
// create a communicator for that new group
MPI_Comm_create(MPI_COMM_WORLD, subgroup, subcomm);
// get my rank in this new group
MPI_Group_rank (subgroup, &subme);
```

You would then use **subcomm** instead of `MPI_COMM_WORLD` whenever you wish to, say, broadcast, only to that group.

9.7 Buffering, Synchrony and Related Issues

As noted several times so far, interprocess communication in parallel systems can be quite expensive in terms of time delay. In this section we will consider some issues which can be extremely important in this regard.

9.7.1 Buffering, Etc.

To understand this point, first consider situations in which MPI is running on some network, under the TCP/IP protocol. Say an MPI program at node A is

sending to one at node B.

It is extremely import to keep in mind the levels of abstraction here. The OS's TCP/IP stack is running at the Session, Transport and Network layers of the network. MPI—meaning the MPI internals—is running above the TCP/IP stack, in the Application layers at A and B. And the MPI user-written application could be considered to be running at a “Super-application” layer, since it calls the MPI internals. (From here on, we will refer to the MPI internals as simply “MPI.”)

MPI at node A will have set up a TCP/IP socket to B during the user program's call to **MPI_Init()**. The other end of the socket will be a corresponding one at B. This setting up of this socket pair as establishing a **connection** between A and B. When node A calls **MPI_Send()**, MPI will write to the socket, and the TCP/IP stack will transmit that data to the TCP/IP socket at B. The TCP/IP stack at B will then send whatever bytes come in to MPI at B.

Now, it is important to keep in mind that in TCP/IP the totality of bytes sent by A to B during lifetime of the connection is considered one long message. So for instance if the MPI program at A calls **MPI_Send()** five times, the MPI internals will write to the socket five times, but the bytes from those five messages will not be perceived by the TCP/IP stack at B as five messages, but rather as just one long message (in fact, only part of one long message, since more may be yet to come).

MPI at B continually reads that “long message” and breaks it back into MPI messages, keeping them ready for calls to **MPI_Recv()** from the MPI application program at B. Note carefully that phrase, *keeping them ready*; it refers to the fact that the order in which the MPI application program requests those messages may be different from the order in which they arrive.

On the other hand, looking again at the TCP/IP level, even though all the bytes sent are considered one long message, it will physically be sent out in pieces. These pieces don't correspond to the pieces written to the socket, i.e. the MPI messages. Rather, the breaking into pieces is done for the purpose of **flow control**, meaning that the TCP/IP stack at A will not send data to the one at B if the OS at B has no room for it. The **buffer** space the OS at B has set up for receiving data is limited. As A is sending to B, the TCP layer at B is telling its counterpart at A when A is allowed to send more data.

Think of what happens the MPI application at B calls **MPI_Recv()**, requesting to receive from A, with a certain tag T. Say the first argument is named **x**, i.e. the data to be received is to be deposited at **x**. If MPI sees that it already has a message of tag T, it will have its **MPI_Recv()** function return the message to the caller, i.e. to the MPI application at B. **If no such message has arrived**

yet, MPI won't return to the caller yet, and thus the caller blocks.

`MPI_Send()` can block too. If the platform and MPI implementation is that of the TCP/IP network context described above, then the send call will return when its call to the OS' `write()` (or equivalent, depending on OS) returns, but that could be delayed if the OS' buffer space is full. On the other hand, another implementation could require a positive response from B before allowing the send call to return.

Note that buffering slows everything down. In our TCP scenario above, `MPI_Recv()` at B must copy messages from the OS' buffer space to the MPI application program's program variables, e.g. `x` above. This is definitely a blow to performance. That in fact is why networks developed specially for parallel processing typically include mechanisms to avoid the copying. Infiniband, for example, has a Remote Direct Memory Access capability, meaning that A can write directly to `x` at B. Of course, if our implementation uses **synchronous** communication, with A's send call not returning until A gets a response from B, we must wait even longer.

Technically, the MPI standard states that `MPI_Send(x,...)` will return only when it is safe for the application program to write over the array which it is using to store its message, i.e. `x`. As we have seen, there are various ways to implement this, with performance implications. Similarly, `MPI_Recv(y,...)` will return only when it is safe to read `y`.

9.7.2 Safety

With **synchronous** communication, deadlock is a real risk. Say A wants to send two messages to B, of types U and V, but that B wants to receive V first. Then A won't even get to send V, because in preparing to send U it must wait for a notice from B that B wants to read U—a notice which will never come, because B sends such a notice for V first. This would not occur if the communication were asynchronous.

But beyond formal deadlock, programs can fail in other ways, even with buffering, as buffer space is always by nature finite. A program can fail if it runs out of buffer space, either at the sender or the receiver. See www.llnl.gov/computing/tutorials/mpi_performance/samples/unsafe.c for an example of a test program which demonstrates this on a certain platform, by deliberating overwhelming the buffers at the receiver.

In MPI terminology, asynchronous communication is considered **unsafe**. The program may run fine on most systems, as most systems are buffered, but fail

on some systems. Of course, as long as you know your program won't be run in nonbuffered settings, it's fine, and since there is potentially such a performance penalty for doing things synchronously, most people are willing to go ahead with their "unsafe" code.

9.7.3 Living Dangerously

If one is sure that there will be no problems of buffer overflow and so on, one can use variant send and receive calls provided by MPI, such as **MPI_Isend()** and **MPI_Irecv()**. The key difference between them and **MPI_Send()** and **MPI_Recv()** is that they return immediately, and thus are termed **nonblocking**. Your code can go on and do other things, not having to wait.

This does mean that at A you cannot touch the data you are sending until you determine that it has either been buffered somewhere or has reached **x** at B. Similarly, at B you can't use the data at **x** until you determine that it has arrived. Such determinations can be made via **MPI_Wait()**. In other words, you can do your send or receive, then perform some other computations for a while, and then call **MPI_Wait()** to determine whether you can go on. Or you can call **MPI_Probe()** to ask whether the operation has completed yet.

9.7.4 Safe Exchange Operations

In many applications A and B are swapping data, so both are sending and both are receiving. This too can lead to deadlock. An obvious solution would be, for instance, to have the lower-rank node send first and the higher-rank node receive first.

But a more convenient, safer and possibly faster alternative would be to use MPI's **MPI_Sendrecv()** function. Its prototype is

```
int MPI_Sendrecv_replace(void* buf, int count, MPI_Datatype datatype,
    int dest, int sendtag, int source, int recvtag, MPI_Comm comm,
    MPI_Status *status)
```

Note that the sent and received messages can be of different lengths and can use different tags.

9.8 Use of MPI from Other Languages

MPI is a vehicle for parallelizing C/C++, but some clever people have extended the concept to other languages, such as the cases of Python and R that we treat in Chapters ?? and ??.

9.9 Other MPI Examples in This Book

- The pipelined prime number finder in Chapter ??.
- Bucket sort with sampling, in Section [10.5](#).

